Optimus - Architecture (HLD)

Version 2.20 — capability and evidence correction, 2026-10-15; reviewed local filing candidate.

Predecessor: Optimus-Cost-Agent-Architecture-v2.19.pdf; SHA-256 427b837ae97db6ed7b8f63529d7dd84fa653265f2c6be1917ad2f00241fca916. v2.19 was filed locally on 2026-10-05 and merged to main through PR #276 (c509fc8, 2026-10-08).

Native carried page bodies originate from Optimus-Cost-Agent-Architecture-v2.18.pdf (SHA-256 0f8725765fecc9a93045fd26630457dfe7112508df164a3ec5bcc55dbc976807), exactly as v2.19 carried them; corrected bodies are replaced in place and the new status section is appended. Every v2.19 page position is accounted for in the correction map.

What this edition changes. For every material capability it separates what is implemented, what has been verified and how, and what is active, at the inspected source 91fd64ac78025efe1076bddb106ff89e6adb2030 (origin/main, 2026-10-15). It changes no accepted requirement, number or historical record. Each correction is listed with its source reference in the correction map filed with the sources (docs/sources/capability-evidence-correction/).

Inspected production facts at 91fd64a. ENFORCEMENT_ACTIVE is False and trusted_snapshot() returns None; no production path constructs ContextAttachment; the shipped alias claude-haiku resolves to anthropic/claude-haiku-4.5; the Gateway web-search provider is Tavily with a Gateway-only key; package/OSV routes still depend on that search configuration; no PyTestArch, maintainability-index, DeepEval, Ragas, PyRIT, Tirith or Vercel dependency, import or test exists; no Rust source exists.

Historical acceptance stays attached to its pins. Offline implementation candidate cf9bb9d (Codex review 2026-10-04); documentation filing 5ee081e; operator acceptance of the offline delivery through 2068366 (2026-10-08); merge c509fc8 (PR #276). The deferred live obligations (Luna route/estimator qualification, genuine summary-quality receipt, real-editor Task 13) remain UNRUN under Plan 12.2 ownership (issue #237). Offline acceptance is not production acceptance, a full CP4 pass or permission to remove the shipped model default.

Authority. Prepared by Claude under the operator's documentation-correction request; Fable 5.1 and Codex review at the plan's checkpoints. This document grants no paid or live calls, activation, push, PR or merge authority, and it was produced without running any product suite.

1. Executive Summary

Conceptual framing adapted from evolutionary software boundaries formulated by Neal Ford, Rebecca Parsons, and Patrick Kua (O'Reilly, 2026). The Optimus-Cost-Agent transitions from a basic token-routing utility to a formal Harness Engineering framework designed to govern non-deterministic generation via deterministic software boundaries. Standard AI agents are fundamentally bounded by the Dreyfus model's skill acquisition ceiling; they cannot brute-force their way past the competent or proficient barrier into true architectural consistency through raw scaling alone. Optimus raises the agent's effective architectural competence through deterministic constraints and feedback by wrapping logical model abstraction inside an opinionated local-first Python ACP server governed by automated, multi-tier architectural fitness functions and explicit request-level cost attribution.

2. Harness Engineering & Context Optimization

To protect the token window from context bloat while systematically mitigating hallucination, the agent structure enforces a strict division between Feedforward Controls and Feedback Sensors:

- **Feedforward Controls (Pre-Generation):** Before a prompt payload hits the downstream model, the Context Optimization Node constructs structural boundaries. The intended controls are AST-based functional slices, static filesystem metadata placed at the head of the prompt window to maximise verified route input caching, and succinct constraints injected in an Architecture Definition Language (ADL) dialect. At the inspected source (91fd64a) the implemented controls are bounded workspace evidence assembly, the Redis structural state store and the optional Context Engine view (§14); an AST slicing component and an ADL constraint injector are specified but not established by inspection (no `ast` import and no ADL grammar under `src/`). See §15.
- **Feedback Sensors (Post-Generation Evaluation):** Once the primary model provider emits a candidate patch, the engine catches the text stream inside a validation loop. It intercepts execution blocks and evaluates them via automated, triggered fitness functions before any source mutations are applied to the active working tree. The implemented gate is the composite fitness runner in `optimus.gates` (`FitnessGateRunner` over composition-supplied `FitnessCheck` callables, run against the shadow workspace by `ShadowWorkspaceMutationRunner`); the set of checks is supplied by the composition, not fixed by this document.

3. Automated Architectural Fitness Functions

Optimus treats system governance not as a passive logging layer, but as an active, objective integrity check on core architectural traits. Phase 1 specifies highly targeted, local validation primitives organised across five operational planes: Scope (Atomic), Automation (Automated), Invocation (Triggered), Return (Static), and Activation (Proactive). These gates intercept code generations mid-flight to fix structural deviations before changes lock into local Git trees. Implementation status: the composite fitness gate runner and shadow-workspace promotion exist and are unit/integration tested; the named architecture-metric checks (PyTestArch, maintainability index/complexity) are intended checks with no dependency, import or test at 91fd64a (§15).

4. Data Governance Plane & Local Storage Boundaries

To support enterprise compliance review, data within the local container is sharply bounded. Unparsed source code is strictly prohibited from entering persistent vector indexes. This is a storage-boundary rule for persistent indexes; it does not by itself establish that no source text is disclosed through prompts, logs, conversation history, telemetry metadata or backups, and generated summaries can also carry sensitive content (§14.3). The storage engine uses standard Redis HASH structures to catalogue structural class signatures, summaries, and relative file paths. Numeric performance histories are channelled separately to RedisTimeSeries (tracking tokens_input, tokens_output, cost_usd) tagged by unique run_id strings with a 30-day retention (2,592,000,000 ms, `TS.CREATE` with `TS.ALTER` fallback in `optimus.telemetry.redis_adapter`).

5. Request-Level Cost Attribution

Target contract after the named trusted-policy / Task 11 / coverage migration as applicable; the dated inactive production baseline is stated on the cover.

5A. Upstream Aggregator Cost Normalization and Single-Key Model

`OPTIMUS_API_KEY` is the only agent-facing credential. It is a local shared secret used to authenticate the agent process to the loopback Optimus Gateway; it is not an upstream vendor key or an Optimus tenant wallet. The Gateway process holds one developer-owned aggregator credential: `OPTIMUS_LOCAL_GATEWAY_PROVIDER_API_KEY`, a Gateway-child-only secret that the launch policy resolves from `.env.gateway`, the OS keyring or the ambient environment with recorded provenance and never projects to the agent child. OpenRouter is the default aggregator. Vercel AI Gateway is an allowed second OpenAI-compatible endpoint by design; no Vercel integration exists at the inspected source (91fd64a).

The developer funds the aggregator account directly. The aggregator supplies access to many models, routes across upstream providers, reports normalized usage and USD cost, and debits one developer-owned balance. The local Gateway preserves the one-key, one-account, one-ledger model by isolating that credential from the LLM-driven agent, enforcing trusted policy and capacity controls; host finite-work bounds remain separate, and recording provider-reported usage.

There is no Optimus-hosted account, prepaid balance, subscription, tenant, org, project wallet, OAuth/device flow, or public Optimus Gateway.

Normalized cost path

Stage	Credential and responsibility
Local agent	`OPTIMUS_GATEWAY_URL` and `OPTIMUS_API_KEY` only
Loopback Gateway	Shared-secret auth, policy, capacity, attribution, aggregator credential isolation
Aggregator	Models, routing, provider-reported billing units and `cost_usd`
Local ledger	Run/session/request/Gateway/provider request attribution

Web search: at the inspected source the Gateway's search provider is Tavily, configured with `TAVILY_API_KEY` in the Gateway process only (`optimus_gateway.providers.build_tool_dependencies`, `TavilyWebToolProvider`); a second, Gateway-held key and balance therefore exist for search today. The accepted target is a dedicated, deterministic OpenRouter plugin request sharing the aggregator credential. That target was measured once by the 2026-07-27 architecture spike (`tools/run_openrouter_web_search_spike.py`, run against OpenRouter directly) and is not implemented in the Gateway. The one-key property holds for model completions today; for search it is a target, not a current fact. Package registry and OSV calls are free public APIs and are not funding paths. OpenTelemetry trace export has no invented per-request charge.

The one-key property for search is explicitly conditional: OpenRouter has deprecated its deterministic plugin, and no deterministic aggregator successor is presently documented. Plugin withdrawal may require a verified-or-fail server-tool design or a standalone search provider with a second key and balance.

Target contract after the named trusted-policy / Task 11 / coverage migration as applicable; the dated inactive production baseline is stated on the cover.

6. Deterministic Data-Flow Architecture (Phase 1 MVP)

The updated system design integrates closed-loop Harness Engineering directly into the orchestration sequence:

1. User Request Initiated (IntelliJ IDEA Client).
2. Ingress over ACP Protocol over StdIn JSON-RPC to the Optimus ACP Server Daemon (Asyncio Pipeline).
3. Read Topology & Structural Memory Maps from Local Redis State Store (HASH Schema Core Pull), which returns active code signatures and cached map assets.
4. Processing inside the Context Optimization Node [FEEDFORWARD CONTROLS]: intended AST Slicing, Polyglot Fallbacks, ADL Constraint Prompt Injection, and Prompt Anchoring for Cache Tuning. At 91fd64a this step is bounded workspace evidence assembly plus the optional Context Engine view (§14); the AST and ADL components are not established by inspection.
5. Emits minimal context prompt payload to the host-selected eligible fixed-model path under the curated registry. Auto classification and automatic escalation remain deferred.
6. **Delivery to the loopback Optimus Gateway -> approved upstream aggregator (OpenRouter by default; Vercel AI Gateway if retained), returning the model output and provider-reported usage and USD cost. Model aliases remain policy inputs, but the agent never selects or contacts a direct provider adapter.**
7. Validation inside the Automated Fitness Function Engine [FEEDBACK SENSORS]: the composite fitness gate (`optimus.gates`) runs the composition-supplied checks against the shadow workspace. PyTestArch and maintainability-index/complexity checks are intended check implementations; none is a dependency, import or test at 91fd64a. On FAIL: loops back to Step [6] while the finite planning bounds permit; on PASS: approves the code patch execution path.
8. Storage inside the Local Storage Engine Persistence Layer via Async TS.ADD Numeric Telemetry Tracking (RedisTimeSeries) and Async HSET Workspace Metrics Serialisation (Redis HASH Architecture Indexes).
9. Cost and usage updates delivered to the ACP client as usage notifications; a dedicated IDE FinOps dashboard panel is a client-side intent, not an Optimus component.

7. Agent Operating Modes & Trust Framework

To guarantee user trust and operational predictability, the control plane implements strict execution state isolation:

- **Plan/Chat Mode:** Advisory-only mode. The agent may inspect context, discuss requirements, propose plans, and produce diffs or snippets for review, but it must not mutate the repository, filesystem, external services, or user/project state. Internal cost, audit, and performance telemetry is append-only and explicitly allowed.
- **Agent Mode:** Execution mode. The agent is write-authorised and empowered to actively modify code, run tools, create files, update tests, and apply patches within approved workspace boundaries. Modifications reach the working tree only through the mediated path: `assert_mutation_allowed` / `MutationGuard`, exact artifact approval and `ShadowWorkspaceMutationRunner`, which promotes shadow changes only after the composition-supplied fitness checks pass. This guarantee covers tool paths that use that runner and is verified by unit and integration tests; it is not a host-wide guarantee over processes outside the agent.
- **Code Generation Scope Classification:** Delineates advisory inline fragments from implementation deliverables based on clear architectural criteria:
  - `INLINE_SNIPPET`: Explanatory text fragments under 15 lines that touch zero core packages and create/delete no files.
  - `PATCH_PROPOSAL`: A bounded diff or file mutation proposal for review.
  - `IMPLEMENTATION_DELIVERABLE`: A multi-file change that requires Agent Mode and all release gates.

The enumeration continues on the next page with the four-value classification the permission layer uses (`MULTI_FILE_CHANGESET` is the implemented impact class in `optimus.guardrails.permissions`); a `CodeGenerationScope` enum is not present as such at 91fd64a.

– **PATCH_PROPOSAL:** Token streams over 15 lines that edit existing code assets exclusively without file structural modifications.

– **FILE_MUTATION:** Single-file generation operations, re-writes, or file deletions that require shell checking or tool alignment.

– **MULTI_FILE_CHANGESET:** High-risk operations touching shared/core architecture modules, crossing distinct directory roots, or altering multi-package configuration environments.

8. Tool Governance & Evidence Acquisition

Optimus treats tool use as policy-driven first, model-requested second: the model may ask for evidence, but the harness alone decides whether a tool is allowed, necessary, and cost-justified. This keeps tool invocation aligned with the same cost-control discipline applied to model routing.

Tool Stack (Phase 1)

Tool Class	Examples	Invocation Style
Local repo discovery	rg, file reader, AST parser, dependency graph, git diff	Deterministic, cheap, first-line
Validation gates	Tests, lint, type check, architecture fitness checks, security scan subset	Deterministic after patch generation
Patch workspace	Shadow apply, diff parser, rollback, working-tree apply	Agent mode only
Cost telemetry	RedisTimeSeries, run metadata hash, pricing snapshot	Always-on internal telemetry
Web evidence	Gateway search + extract (Tavily-backed)	Policy-triggered, read-only
Package/version metadata	PyPI, npm, Maven Central, GitHub releases, OSV.dev	Only when dependency/version/security task requires it
External execution	Shell, build, test, package install	Agent mode only, gated

Deterministic Invocation Policy

A ToolInvocationPolicy matrix maps task signals to permitted tool classes, so "search the web" is never a casual model impulse. Representative triggers:

- User explicitly asks for current/latest/external facts → web search.
- Task mentions library/API/framework behaviour not present in the repo → official-doc web search.
- Task touches dependencies → package registry lookup.
- Task touches security/CVEs → OSV or advisory lookup.
- Task asks for a local code change → local repo search first, no web by default.
- Model makes an architectural claim without inspected evidence → file inspection or Assumption Ledger entry.
- Patch generated → shadow apply + fitness gates.

• Fitness gate fails -> reflection or targeted local inspection.

• Plan/Chat mode active -> read/search tools only, no mutation tools.

Each web-evidence request carries an explicit reason code (e.g. API_DOCS_OUTDATED, CURRENT_FACT, SECURITY_ADVISORY, USER_REQUESTED, PACKAGE_VERSION) plus an allowed-domain list, result cap, and search-depth setting, so external calls remain auditable and cost-bounded rather than ad hoc.

Web evidence acquisition is wrapped behind Optimus-owned typed tools rather than exposing a third-party search API directly to the model. This lets the harness enforce allowed domains, timeouts, retries, result caps, and cost telemetry uniformly. Key rule: local evidence first, external evidence only when policy-triggered, mutation only after mode and fitness gates pass.

9. Adaptive Agent Execution Strategy & Rigor Policy

Designed strategy policy: Optimus applies adaptive execution strategies to balance cost, rigor, and reliability. The agent selects Direct Execution, Plan-and-Execute, ReAct, or Reflection based on task ambiguity, mutation risk, and validation outcomes. Reflection and deep planning are not default behaviours; they are exception-triggered circuit breakers invoked only when risk signals, failed validation, or insufficient evidence require additional rigor.

Low-Cost Hallucination Controls

• Evidence-Bound Decisions: No architectural claim may be promoted to an implementation

decision unless backed by inspected context or recorded as an explicit entry in a machine-readable Assumption Ledger.

• Rigor Budget Matrix: Bounded request tiers (LOW, MEDIUM, HIGH) prevent the agent from

spending tokens on ceremonial planning loops for simple edits:

– LOW: Cap at 5 tool calls, 0 reflection passes, min(262144, verified route window)-token total request ceiling including output; eligible fixed model.

– MEDIUM: Cap at 10 tool calls, 1 reflection pass, min(262144, verified route window)-token total request ceiling including output; eligible fixed model.

– HIGH: Cap at 15 tool calls, 3 reflection passes, min(262144, verified route window)-token total request ceiling including output; eligible fixed model.

• Trigger-Based Reflection: Reflection initialises only when validation loops trip a failure

signature or code touches shared/core architecture modules, preventing quiet token-doubling on every nominal request.

• Assumption Ledger: Bypasses token-heavy text summaries by generating an explicit JSON

ledger identifying unverified assumptions, confidence thresholds, and specific validation files needed to clear uncertainty.

• The Escalation Ladder: Bounds agent tool invocation paths to escalate strictly through a

cost-minimising chain: Provided Context -> File Inspection -> Plan -> Execute -> Validate -> Targeted Reflection -> User Escalation.

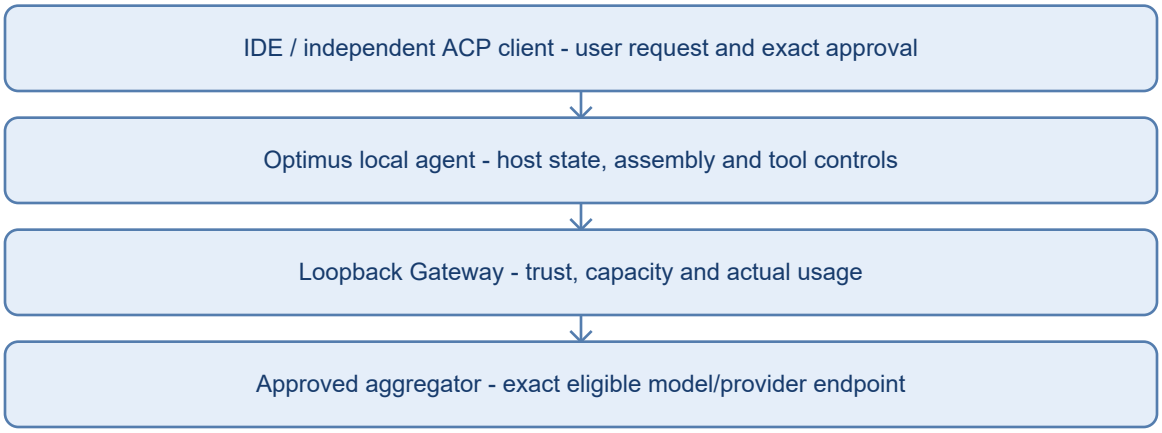
Implementation status (91fd64a): the strategy names, the LOW/MEDIUM/HIGH rigor tiers with their tool-call and reflection caps, and the Assumption Ledger are design policy. No `RigorLevel`, `ExecutionStrategy` or `AssumptionLedger` type exists under `src/`; telemetry carries `rigor_level` and `generation_scope` as string fields. The active runtime bounds are the ACP planning limits (3 planning turns by default, `OPTIMUS_MAX_PLANNING_TURNS`; 30-minute wall clock; repeated-failure limit 2 for ACP planning), the Gateway per-tool call cap (5 calls per tool per run by default, `OPTIMUS_GATEWAY_TOOL_MAX_CALLS_PER_TOOL`) and the finite loop controls referenced in §13. The min(262144, verified route window) request ceiling is implemented in `optimus_model_policy`, with production enforcement inactive (§14.4).

10. Architectural Control Flow

This section provides visual reference diagrams for the two primary control flow concerns in Optimus: where the system boundaries sit, how the governance loop operates end-to-end.

10.A System context diagram

The IDE, local agent, loopback Gateway, Redis, repository and optional Phoenix collector belong to one developer environment. Agent and Gateway are separate processes; the agent receives only Gateway URL/shared secret and cannot read the Gateway aggregator credential.



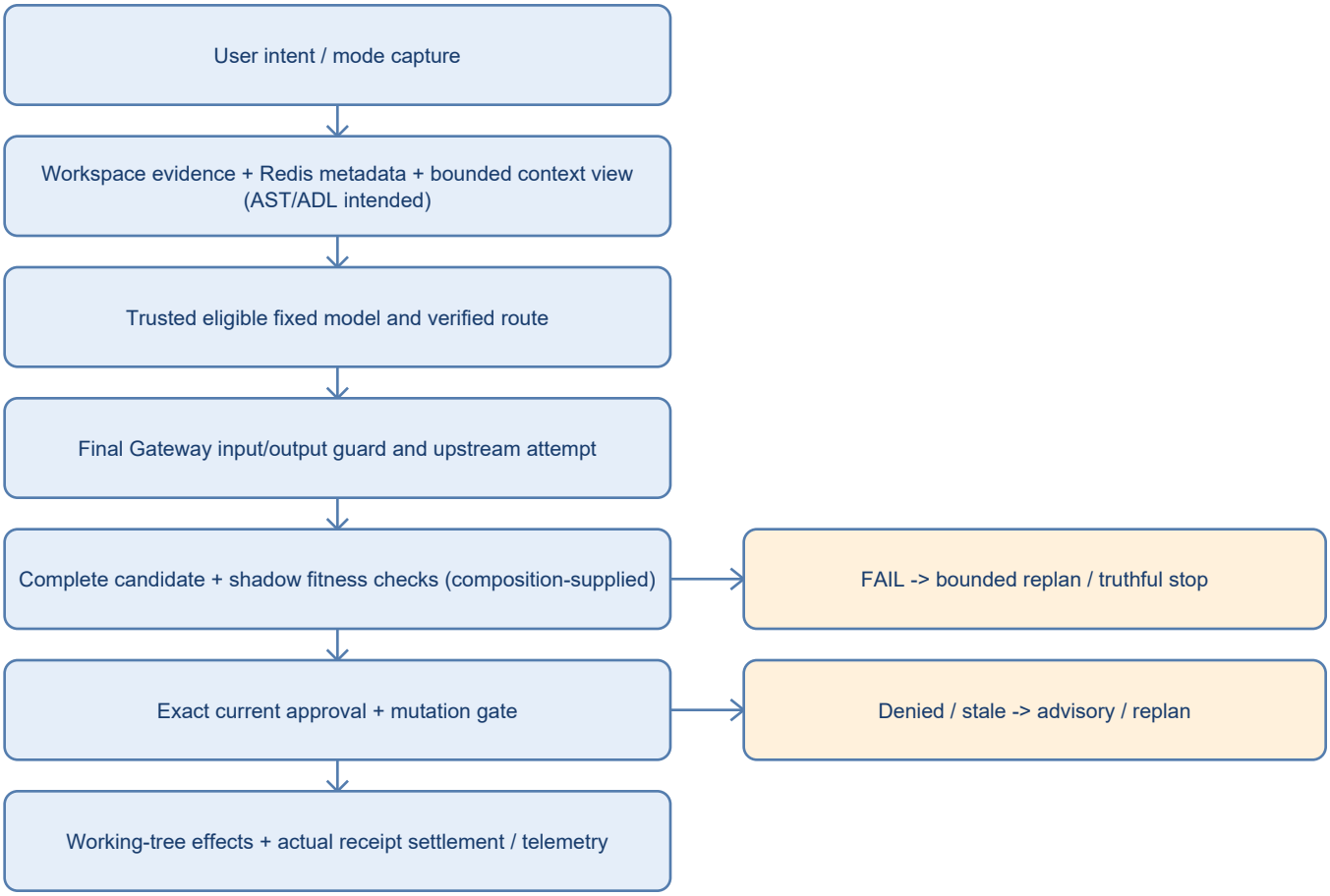
Side edge	Boundary
Agent -> repository	Current eligible workspace evidence and controlled approved edits
Agent / Gateway -> Redis	Existing state and usage ledger; no new context-engine store
Gateway -> Phoenix	Configured OTLP export; no invented billable charge
Gateway -> PyPI/npm/Maven	Independent public package APIs
Gateway -> OSV	Independent advisory service
Gateway -> OpenRouter	Default approved aggregator/model; the separately authorized search path is a target (see Tavily)
Gateway -> Tavily	Implemented web-search provider at 91fd64a with its own Gateway-only key; replacement by the aggregator plugin path is pending (§5A)
Gateway -> Vercel	Design option; no integration at the inspected source

No MCP endpoint is shown or implied. Client-supplied MCP trust/execution remains local. WSL2 agent and Windows-host Gateway do not share loopback; Phase 1 requires the same network namespace. Figure 10.A preserves that boundary and retires the product-budget label. Separate processes on one loopback network namespace isolate the aggregator credential from the agent process; they do not by themselves confine either process from the rest of the host, and whole-host confinement is not claimed.

Target contract after the named trusted-policy / Task 11 / coverage migration as applicable; the dated inactive production baseline is stated on the cover.

10.B Governance loop diagram

Every generated executable candidate remains subject to feedforward context/trust constraints and feedback/approval gates. Haiku triage and a hard-coded GLM/Pro route are replaced by the explicit curated fixed-model path.



A failed fitness gate supplies targeted failure context only while the configured planning/time/repeated-failure bounds permit another round; otherwise report the stop truthfully. Plan/Chat never enters the mutation path. Current interim ACP bounds remain three planning rounds, 30 minutes and two repeated failures. No Auto classifier, product dollar stop or new recovery graph is activated. Figure 10.B retains the existing feedforward/feedback distinction and closes the approval/effect loop.

Target contract after the named trusted-policy / Task 11 / coverage migration as applicable; the dated inactive production baseline is stated on the cover.

10.C Phase Evolution Diagram

Optimus is designed as a three-phase architecture. Phase 1 is mandatory before any later phase.

PHASE 1 -> Python Local Agent + Optimus Gateway Current - Mandatory before Phase 2

Release Gate: the agent runs with only `OPTIMUS_GATEWAY_URL` and `OPTIMUS_API_KEY`; no upstream key is resolvable in the agent process. The separate loopback Gateway receives only its approved aggregator key and optional infrastructure configuration.

PHASE 2 -> Rust Core Rewrite (proposed; no Rust source exists in the repository at 91fd64a) Trigger: Python performance ceiling or team scaling

PHASE 3 -> Agentic Mesh Topology (proposed) Trigger: Multi-repo / multi-team deployment

See: Optimus-Architecture-Roadmap-v11.pdf, which classifies each phase as implemented, approved requirement or proposed.

10.D Where Cost Control Happens

Cost control is not a single checkpoint - it is enforced at four distinct points in the request lifecycle, each with a different mechanism.

Control Point	Mechanism	What It Prevents
1. Pre-prompt (Feedforward)	Bounded workspace evidence and context view today; AST slicing and ADL constraints intended, not established	Unnecessary tokens sent to expensive models.
2. Router (Triage Gate)	Explicit eligible fixed model from trusted registry; Auto classifier remains dormant	Over-routing simple tasks to ineligible or unnecessarily expensive route.
3. Rigor Budget (Runtime)	Finite work bounds: planning turns, wall clock, repeated failures and the per-tool call cap (rigor tiers are design policy, §9)	Token-doubling on ceremonial planning loops.
4. Post-call (Attribution)	Gateway usage object -> EvidenceLedger -> RedisTimeSeries	Silent cost accumulation; enables per-run audit.

10.E Where Hallucination Control Happens

Hallucination is controlled through a defence-in-depth stack. No single gate is sufficient; all five layers are intended to be active simultaneously. §15 states which layers are implemented at 91fd64a.

Layer	Mechanism	What It Catches
1. Context Constraint	Bounded workspace evidence and context view (ADL rules and AST slices intended, not established)	Irrelevant context that confuses the model.
2. Evidence Gate	Evidence-bound decisions as a design rule; no Assumption Ledger artifact at the inspected source	Unverified claims promoted to implementation decisions.
3. Tool Policy	ToolInvocationPolicy blocks casual web calls; reason codes required	Model hallucinating external facts without evidence.
4. Fitness Functions	Composite fitness gate (`optimus.gates`); PyTestArch and static-metric checks intended, not integrated	Structurally invalid or low-quality changes.
5. Reflection Circuit	Bounded replanning with failure context within finite work limits (reflection passes are design policy)	Repeated failure without targeted replanning.

11. Optimus Gateway - Phase 1 Local Process Boundary

The Optimus Gateway is a deterministic loopback process run by the developer alongside the local agent. It is not a hosted Optimus service, tenant control plane, subscription product, or central wallet.

Gateway responsibilities

- Bind to strict loopback and authenticate the agent with a local shared secret.
- Isolate the developer's aggregator credential in the Gateway process: the launch policy resolves it from `.env.gateway`, the OS keyring or the ambient environment with recorded provenance and projects it only to the Gateway child.
- Route explicitly eligible model IDs through the surviving OpenAI-compatible transport after trusted policy composition (trusted-policy enforcement is built and inactive at 91fd64a: `ENFORCEMENT_ACTIVE = False`).
- Enforce trusted model/tool policy, domain rules, request call caps, provenance and complete request capacity when enabled. Host count/time/failure limits remain separate; product dollars are accounting/alerts after the released migration.
- Return provider-reported usage and cost in a validated GatewayUsage envelope.
- Broker web search (Tavily provider at 91fd64a; the deterministic aggregator plugin is the target), bounded extract, package lookup, and OSV advisory. Independence of the package/OSV routes from search configuration is a requirement not yet met: one `GatewayToolDependencies` bundle is built, and it is `None` when `TAVILY_API_KEY`, the tool Redis URL or the allowed-domain list is missing (LLD §9C.4).
- Accept structured trace ingress, map it to OpenTelemetry, and export OTLP.
- Persist usage attribution across run, session, request, Gateway request, and provider request IDs.

Process-scoped configuration

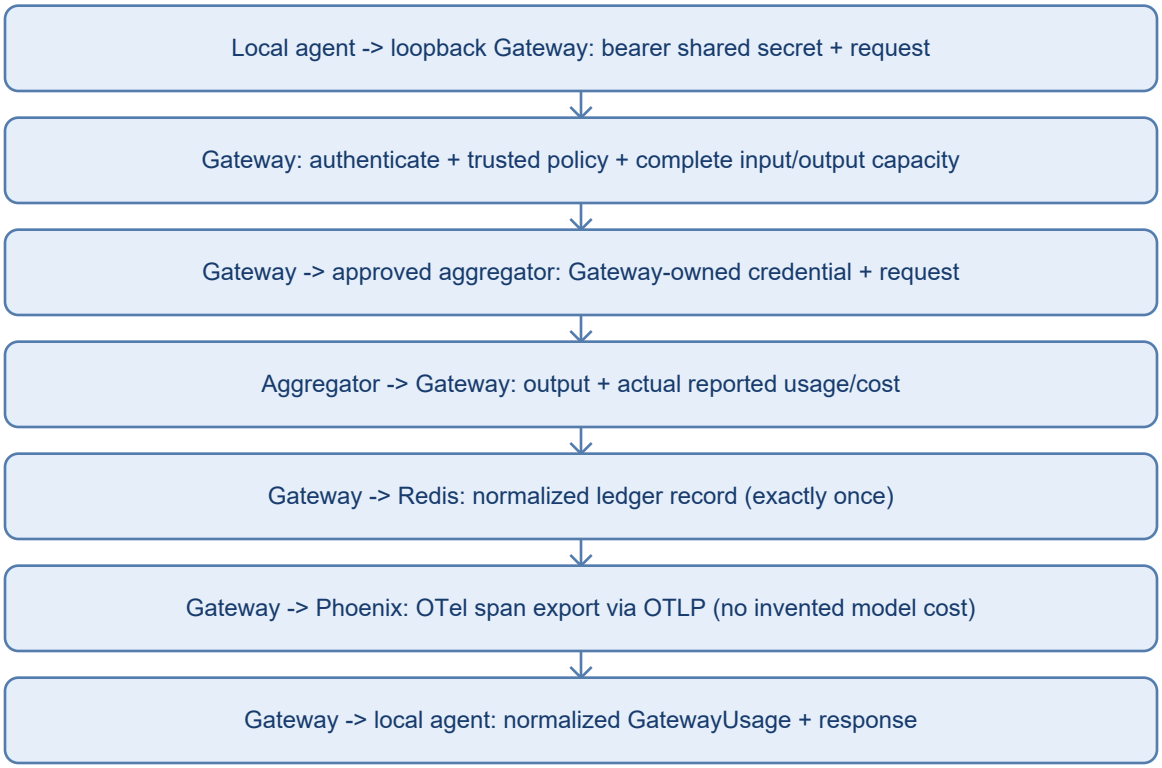
Agent process environment	Gateway process local configuration
`OPTIMUS_GATEWAY_URL=http://127.0.0.1:8765`	`OPTIMUS_LOCAL_GATEWAY_PROVIDER=openrouter`
`OPTIMUS_API_KEY=`	`OPTIMUS_LOCAL_GATEWAY_PROVIDER_API_KEY=`
	`TAVILY_API_KEY=` (Gateway-only; current search provider)
No provider, search, or OTel credentials	`OPTIMUS_LOCAL_GATEWAY_SHARED_SECRET=`
Loopback URL only	allowed domains, Redis URL, optional OTLP endpoint
No hosted-origin override	no tenant profile, Vault, or non-loopback mode

Run the agent and Gateway in the same network namespace. A Windows-host Gateway is not loopback from an agent running inside WSL2.

Target contract after the named trusted-policy / Task 11 / coverage migration as applicable; the dated inactive production baseline is stated on the cover.

11.1 Gateway request / response sequence

The actual request path retains the four actors: local agent, loopback Gateway, approved aggregator and Redis/Phoenix. The accepted product migration replaces the old monetary budget precheck with trust, complete capacity and finite work controls.



Sequence boundary	Preserved behavior
Agent to Gateway	One authenticated request over the strict loopback boundary. The aggregator credential never enters the agent process or response.
Gateway to aggregator	Only the configured approved aggregator with Gateway-owned credential. Final complete-request and output enforcement applies to every actual attempt.
Aggregator result	Parse output/true finish status and actual reported usage/cost. Unknown cost is explicit, never manufactured as zero.
Ledger and trace	Persist normalized receipt exactly once and export sanitized OTel spans independently; trace export is not a billed inference request.
Search	A separate authorized tool request (Tavily provider at 91fd64a; the aggregator-plugin annotation path is the target). Package and OSV routes are required not to depend on search configuration; at 91fd64a they are constructed in the same dependency bundle and are unavailable when `TAVILY_API_KEY` is missing (LLD §9C.4).

Independent evaluation dollar caps remain scoped to their authorized evaluations. Product dollar stops were removed at cf9bb9d and merged to `main` via PR #276 (`c509fc8`, 2026-10-08); production context enforcement remains inactive at 91fd64a. Evaluation ceilings remain scoped to independently authorized evaluations.

11A. OpenTelemetry trace observability

Phase 1 uses OpenTelemetry spans and OTLP as the vendor-neutral trace contract across planning, Gateway calls, tool invocation, validation, retries and final response generation. The local agent sends authenticated structured trace ingress to the Gateway; it validates/redacts fields, maps them to OTel spans/events and exports OTLP. Arize Phoenix is the documented local default. Any OTLP-compatible backend may replace it without changing Optimus instrumentation; Langfuse may be considered for team-scale deployments.

Trace export records operational telemetry, not an invented billable provider request. No allocated/amortized observability cost_usd is added to the usage ledger. Infrastructure cost is an operator concern outside per-request accounting.

Required attributes retain run, session, request, Gateway/provider request, execution mode, generation scope, model/provider, cache, cost_usd, billing units, policy/tool, validation, retry and failure attribution. Secrets are redacted before export.

12. Testing and quality gates

Concern	Tooling	Role in gates
Code coverage	coverage.py, pytest-cov	Implemented and enforced in CI at 91fd64a: `tools/check_product_coverage.py --run-full-suite` with `tools/product-coverage-groups.json` (Optimus group, context_engine and optimus_model_policy at 80%; Evidence Handoff and the aggregate informational) followed by `tools.testing.coverage_products validate`. The commit hook runs the coverage-free commit profile (`tools.testing.run_profile --profile commit`). Test Strategy v1.9 §8A governs. Safety-critical modules must not regress.
Agent quality	DeepEval, Ragas	Intended; no dependency, import or test at 91fd64a; UNRUN
Adversarial/red-team	PyRIT	Intended; no dependency, import or test at 91fd64a; UNRUN. The prompt-injection and MCP-trust fixtures in `tests/unit/guardrails` are offline positive controls for specific guards, not a red-team campaign
Trace observability	OpenTelemetry + OTLP; Phoenix default	Implemented (`optimus_gateway.observability`); the live Phoenix tier is the `requires_phoenix` marked tier and is UNRUN unless a collector is present

OTel/OTLP trace validation is tracked separately from production-code coverage. LangSmith is not part of the architecture or dependency set.

13. Agent Execution Safety & Guardrails

Agent execution safety is enforced through a layered guardrail model that wraps every tool call between the moment the agent decides to act and the moment the action takes effect: mode constraints (§7), a permission policy (allow/deny lists with a human-approval path), a pre-tool guard that deterministically validates shell, file, MCP, and network calls, sandboxed execution (the shadow workspace; OS-level sandboxing is not claimed), post-tool audit against the Gateway cost and trace ledgers (§5A, §11, §11A), and commit/CI gates (§12). The same plane governs two execution patterns — bounded goal-driven agent loops and curated workflow skills — so that long-running or procedure-driven work inherits the identical constraints and the same Gateway trust, capacity and accounting policy.

The premise is not that the agent is malicious but that it is a reward-hacky problem solver: under pressure to reach "done" it will reach for chmod 777, curl … | sh, force-push to main, or commenting out a failing assertion. Guardrails remove those shortcuts from its reach. Most of the model is deterministic and zero-token; a cheap, finitely bounded classifier routed through the Optimus Gateway is consulted only for borderline cases, never on the hot path of an already-allowed or already-denied call.

The detailed policy model, component contracts, and control-flow diagram are maintained in the companion specification Agent Execution Guardrails & Workflow Strategy v1.4 (control- plane diagram in its §1; LLD component contracts in its §10; test categories in its §11). The corresponding implementation contracts are specified in LLD §12 and the test categories in Test Strategy §14. This section is a cross-cutting reference only.

Implemented at 91fd64a with unit tests under `tests/unit/guardrails`, `tests/unit/agent`, `tests/unit/loops`, `tests/integration/loops` and `tests/unit/skills`: `PermissionPolicy`, `PreToolGuard`, `CommandSafetyValidator` (in-process; Tirith is not a dependency), `ConfigTrustScanner`, `MCPTrustRegistry`, `GoalLoopController` (`optimus/loops/controller.py`) with `LoopBudgetPolicy`, and `SkillRegistry` with `SkillTrustPolicy`. The classifier described here is a reserved architecture capability, not an active production route. Host finite work controls and Gateway capacity/policy enforcement have separate owners. Coverage policy is authoritative in Test Strategy v1.9 §8A: Optimus product group, context_engine and optimus_model_policy have separate 80% gates; all-source coverage is informational.

14. Optional Context Engine and Plan 12 migration

14.1 Product boundaries

`context_engine` is an optional, extractable in-process Python package. It receives immutable sanitized ordinary turns, exact host-issued execution/approval projections, strategy parameters, limits, revision identity and an injected maintenance callback. It owns derived views and checkpoints only. It imports no Optimus host, Gateway, security, evidence-handoff, workspace, Redis or provider implementation. Verified by import inspection at 91fd64a: `src/context_engine` imports only the standard library, so the zero-dependency property belongs to `context_engine` alone; `optimus_model_policy` additionally imports PyYAML and pydantic.

Optimus owns canonical storage, current prompt, exact authority, turn lifecycle, current workspace resolution, policy capture, model-facing assembly and notices. `optimus_model_policy` is neutral shared policy code; host and Gateway independently consume the trusted immutable snapshot. The Gateway alone performs approved upstream calls and enforces final request capacity for each actual attempt. Existing ledger/telemetry owners account for every attempt.

The local Gateway, shared-secret boundary, independent client-MCP trust controls, public tool dependencies and OTel/OTLP topology remain as specified in earlier sections. Plan 12 adds no Gateway MCP broker, provider key in CORE or generalized plugin platform.

14.2 Canonical source and selected views

Without the engine, retain full canonical history and inclusive 524,288-byte admission, its 80% storage warning, and the already accepted history-order/display exceptions. An attached engine may use the released offline 1,048,576-byte source class; production attachment still requires its own activation. Source size is distinct from provider request capacity and the context meter. A prompt-time 131,072-byte record reservation is not a cap on the eventual reply or entire turn.

Selection and packing are deterministic; only the summary text is generated. Compaction and Hybrid invoke the injected summarizer callback; Sliding never does. Compaction is the default. It retains a newest whole-turn exact tail and summarizes earlier ordinary history. Hybrid optionally retains the first complete turn as an exact anchor, uses a larger exact tail and summarizes the middle; deduplicate overlapping head/tail turns. Sliding takes the newest complete turns that fit, makes zero summary calls and does not skip an oversized newest turn to recover older ordinary history. All three retain exact protected state. Derived omission never deletes canonical history.

The proposed cheap/ultra-cheap history allowance is the smaller of 131,072 tokens and 80% of usable input, further constrained by final packing after protected state. It is a ceiling on history sent, not a threshold that delays summarization. The released offline summary cap is 8,192 tokens; compaction tail is 32,768 tokens, hybrid anchor 16,384 tokens and hybrid tail 65,536 tokens. Thus compaction's ordinary tail plus summary is at most 40,960 tokens; hybrid's head, tail and summary at most 90,112, before exact protected facts and other material. These are adopted offline test values carried by the reviewed test profiles and fixtures (for example `optimus_model_policy/test_profile_qualification.yaml` output reserves 32768/8192), not shipped production defaults; production defaults and route activation remain held.

Whole-turn summarization remains the simplest accepted design direction. If an indivisible older turn cannot fit maintenance, report the reason and the filed recovery choices; do not split it, silently switch strategies or discard ordinary text inside compaction/hybrid. Sliding can omit even all ordinary history when its newest turn does not fit. Any unavailable view at <=524288 projected source bytes may use full-history fallback when its complete final request fits, including an oversized turn or missing summarizer. Above that floor it refuses readably with OPEN disposition; the refusal is recoverable and introduces no monetary latch or new fallback mechanism. Availability depends on the actual history, estimator and bounds. Within-turn splitting and delayed-trigger/call-frequency optimization remain deferred.

14.3 Authority, selection and continuity

Model-authored history and generated summaries are inert. A claim that a change ran, a test passed or an approval exists cannot create host authority. Preserve every committed turn's exact outcome/effect projection and host-issued approval facts, independently of ordinary history selection. Historical approval describes an earlier decision and grants no fresh execution.

Exact retained earlier plans/replies may supply ordinary file/skill selection hints. Resolve file references against the current eligible workspace and assemble current bounded content. A resolved path does not verify an old diagnosis. Generated summary text does not directly select files or skills. A reference appearing only in omitted history may be unavailable to selection; do not claim full-history retrieval after compaction.

Model changes preserve canonical conversation and the strategy-defined view. They do not trigger a paid whole-chat factual audit. Use existing targeted freshness checks for selected file data and controlled edit targets. Preserve approval/version checks; no background watcher, durable claim database, semantic pinning or repository-wide rescan is introduced. These controls do not eliminate races with unrelated external writers.

14.4 Capacity and policy

Every final model request uses `effective_total = min(262144, verified_route_window)`, `usable_input = effective_total - verified_output_reserve`, and a complete-input estimate that includes messages, templates, workspace/tool/evidence material and framing. Admit equality; count the actual enforced output cap once, including verified reasoning semantics. Recompute at the final Gateway boundary for every actual provider attempt. When a model has several permitted endpoints, the applicable smallest verified capacity governs. The estimator is a conservative verified byte-ratio estimate (`ceil(r * UTF8_bytes)` with non-negative overhead), not exact tokenization. A pre-call refusal dispatches no upstream attempt for that rejected request and remains readable/recoverable; earlier maintenance attempts can still have receipts.

D7's proposed named exception retains inclusive 524288-byte canonical storage/admission and its warning, but does not guarantee that full history fits a provider request. Both absent-engine and attached full-history fallback still undergo the final complete-request guard; overflow gives the accepted readable OPEN refusal without answer/planning dispatch. Do not lower the source cap or exempt capacity. The release implements this disclosed exception offline; its verbatim D7 record is filed in ADR-018 from registered ADR-011 source S4 (decision 2026-10-04, filed 2026-10-07). Exact per-route tokenizers remain deferred; route-specific byte-ratio safety requires normalization/pre-tokenization/framing proof and cannot be inferred from samples. Unknown/expanding behavior remains ineligible.

The curated registry retains explicit role assignments, supported capabilities, exact endpoints, provenance and cheapest-per-token ordering among eligible assignments. Crossed prices need the accepted blend contract; no hidden scoring replaces policy. Contributor routes keep their filed disclosure. Haiku is excluded from the curated migration target. This does not claim active enforcement or a shipped-default change; normal bootstrap remains inactive until separately released. At 91fd64a: `ENFORCEMENT_ACTIVE = False` and `trusted_snapshot()` returns `None` (`optimus_model_policy/binding.py`); the shipped alias `claude-haiku` resolves to `anthropic/claude-haiku-4.5`. Auto classification, reviewer/depth behavior, savings-report surfaces and ADR-009's future recovery mechanism are not activated by this migration.

14.5 Costs and execution

The normative product policy after the accepted D6/governed migration has no dollar-stop predicate, hidden summary reserve, positive-dollar constructor requirement, remaining-dollar planner prompt or infinite sentinel. It keeps actual provider-reported costs, exactly-once settlement, notices and explicitly configured alerts. Missing cost remains unknown, never zero; keep known subtotals and incomplete-accounting attribution. Do not automatically retry an uncertain attempt or freeze the session. A later intentional request still uses normal capacity, trust and authorization controls.

Finite work controls remain: current defaults of three Agent planning rounds, 30 minutes and two repeated failures, single-call Chat, bounded maintenance and explicit repacks. Goal loops retain their own finite count/time/failure controls. Independently authorized evaluation/test budgets remain real monetary ceilings. D6's requirement is preserved: there is no product dollar stop. Product dollar stops and remaining-dollar prompts were removed at cf9bb9d and merged to `main` via PR #276 (`c509fc8`, 2026-10-08). Independently capped evaluation remains capped, and such separately authorized evaluation spend caps are not product cost stops. Production context enforcement remains inactive.

14.6 Evidence and release

Implementation checkpoints, measured coverage and exact acceptance/provenance are recorded in the companion closure records. Architecture targets do not promote historical evidence to changed code or establish current gate enforcement.

Complete product activation requires accepted measured policy, supported route/estimator/output facts, an exact qualified summary receipt, production-base effect/cancellation proof, trusted launch composition, finite repack behavior and real-client evidence. There is still no production constructor of ContextAttachment (verified at 91fd64a: no `src/` path constructs it, and its docstring records this); the reviewed named test composition supplies it only within its approved test process. Task 13 may use a separately approved ephemeral test composition through explicitly implemented and reviewed process-scoped dependency injection; that proves the named test configuration, not production bootstrap activation. v2.19 and the offline implementation were merged via PR #276; that merge activated nothing. Auto classification, the final reviewer step, the savings report and package publication keep their own implementation and activation status (§15).

Once enforcement is active, capacity-specific text is selected only for INPUT_EXCEEDS_CAPACITY. The 80% request warning uses complete request size, while the separate storage notice preserves the source floor. A local Gateway dispatch reading is not proof of upstream delivery.

15. Capability, verification and activation status (correction of 2026-10-15)

This section records, for the material capabilities this document names, three independent facts at the inspected source `91fd64ac78025efe1076bddb106ff89e6adb2030` (origin/main, 2026-10-15): whether the capability is implemented, how far it has been verified, and whether it is active. The fields are independent: an implemented guard can be offline-verified and production-inactive; an approved roadmap requirement is not an unapproved idea, but neither is it implemented. A negative repository search supports "integration not established", not a claim that a tool has never been used anywhere. Accepted requirements and historical evidence are unchanged by this section; the correction map filed with the sources (`docs/sources/capability-evidence-correction/correction-map.md`) lists every change with its source reference.

Verification vocabulary. *Code inspection* means the capability was found or not found in `src/`, `tests/`, `tools/`, `pyproject.toml` and `uv.lock` at the pin. *Offline-verified* means a sealed full-suite run on a clean candidate without live dependencies. *Live-verified* means real external dependencies under the named marked tier. *UNRUN* means a defined check that has not been executed. *UNKNOWN* means the fact could not be established.

Capability	Implementation	Verification	Activation
Loopback Gateway with shared-secret authentication (§5A, §11)	Implemented (`optimus_gateway.server`, `optimus/config/gateway.py`)	Unit and integration tests at the pin; sealed offline suites at cf9bb9d	Active
Aggregator credential isolation (§5A, §11)	Implemented: `OPTIMUS_LOCAL_GATEWAY_PROVIDER_API_KEY` is a Gateway-child-only secret resolved by the launch policy with provenance	Unit tests of launch policy and launch gate	Active
OpenAI-compatible upstream transport, OpenRouter default (§6, §11.1)	Implemented (`optimus_gateway/upstream_client.py`)	Unit tests; `requires_gateway` live tier exists	Active
Vercel AI Gateway second endpoint (§5A)	Specified; not established by inspection (no reference in `src/`)	None	Inactive
Web search provider (§5A, §8)	Implemented: Tavily (`TavilyWebToolProvider`, Gateway-only `TAVILY_API_KEY`)	Unit tests; live tiers when configured	Active when configured
Deterministic OpenRouter plugin search sharing the aggregator credential (§5A)	Specified target; not implemented in the Gateway	Spike-measured once, 2026-07-27, by `tools/run_openrouter_web_search_spike.py` outside the Gateway	Inactive
Package and OSV routes independent of search configuration (§8, §11)	Specified; not implemented: one dependency bundle, `None` without the Tavily key, tool Redis URL and allowed domains	None	Not applicable
Bounded HTTPS extract with provenance (§8)	Implemented	Unit tests; live tier when configured	Active when configured
Provider-reported usage ledger and exactly-once settlement (§5A, §11.1)	Implemented (`optimus/usage`, `optimus/evidence`, `optimus/telemetry/redis_adapter.py`)	Unit tests; `requires_redis` tier; sealed offline suites	Active
OpenTelemetry/OTLP export to Phoenix (§11A)	Implemented (`optimus_gateway/observability.py`)	Unit tests; `requires_phoenix` live tier UNRUN without a collector	Active when configured

Capability	Implementation	Verification	Activation
Mutation guard, shadow workspace and composite fitness gate (§2, §7)	Implemented (`optimus/gates`, `optimus/runtime`)	Unit and integration tests	Active for mediated tool paths
PyTestArch and maintainability-index/complexity checks (§3, §6, §10.E)	Not established by inspection: no dependency, import or test	UNRUN	Inactive
AST slicing and ADL constraint injection (§2, §6, §10.D)	Not established by inspection: no `ast` import, no ADL grammar under `src/`	None	Inactive
Rigor tiers, execution strategies and Assumption Ledger (§9, §10.D, §10.E)	Specified; no `RigorLevel`, `ExecutionStrategy` or `AssumptionLedger` type	None	Inactive; active bounds are 3 planning turns, 30 minutes, 2 repeated failures, 5 calls per tool
Permission policy, pre-tool guard, shell validator, config scanner, MCP trust registry (§13)	Implemented (`optimus/guardrails`)	Unit tests (`tests/unit/guardrails`)	Active
Borderline permission classifier (§13)	Specified, reserved	None	Inactive
Bounded goal loops (§13)	Implemented (`optimus/loops`, `optimus/agent/planning_loop.py`)	Unit and integration tests	ACP planning bounds active; goal-loop activation from ACP not granted
Curated workflow skills (§13)	Implemented (`optimus/skills`)	Unit tests	Active
Context Engine strategies, checkpoints and whole-turn packing (§14)	Implemented (`src/context_engine`, standard-library only)	Offline-verified at cf9bb9d (Windows 6830 passed, WSL 6641 passed, exit 0); unit tests at the pin	Production attachment inactive: no production constructor of `ContextAttachment`
Trusted registry snapshot and complete-request capacity guard (§14.4)	Implemented (`optimus_model_policy`)	Offline-verified at cf9bb9d	Inactive: `ENFORCEMENT_ACTIVE = False`, `trusted_snapshot()` returns `None`
D6 accounting: no product dollar stop, unknown cost stays unknown (§14.5)	Implemented at cf9bb9d, merged via PR #276	Offline-verified; former strict xfails now pass	Active
Luna route/estimator qualification, genuine summary-quality receipt, real-editor Task 13 (§14.6)	Prerequisites defined; profile A admits no model	Deferred, UNRUN (operator disposition 2026-10-08)	Held; owner Plan 12.2 (issue #237)
Auto classification and automatic escalation (§6, §10.D)	Proposed (ADR-007, status Proposed); not implemented	None	Deferred under existing owners
Final reviewer step (ADR-008) and net savings report (ADR-012)	Reviewer step proposed (ADR-008, status Proposed); savings report specified (ADR-012 accepted with open items, issue #248); neither implemented	None	Deferred under existing owners
Product coverage gate (§12)	Implemented (`tools/check_product_coverage.py`, `tools/product-coverage-groups.json`)	CI-enforced (`guardrails.yml`); measured at cf9bb9d	Active in CI

Capability	Implementation	Verification	Activation
DeepEval, Ragas and PyRIT evaluations (§12)	Not established by inspection: no dependency, import or test	UNRUN	Inactive
Phase 2 Rust supervisor and Phase 3 mesh (§10.C)	Proposed (Roadmap v11); no Rust source in the repository	None	None

Evidence anchors. `cf9bb9d73ac6c8c3be508966245e0db7cdaaa503` is the historical offline implementation candidate reviewed by Codex on 2026-10-04; `5ee081e` filed the documentation; `2068366` is the boundary through which the operator accepted the offline implementation and documentation on 2026-10-08; PR #276 merged that work as `c509fc8` on 2026-10-08. The acceptance scope and deferred obligations are in `docs/superpowers/reviews/2026-10-04-plan-12-2-cp4-offline-review-disposition.md` and `2026-10-08-plan-12-2-cp4-offline-delivery-disposition.md`; source custody is in `docs/sources/plan-12-2-context-engine-closure/README.md`. Historical passes are not transferred to `91fd64a`. At the filing state this correction's documentation checks passed; the coverage-free commit-profile hook (`run_profile --profile commit`) selected a broad fallback of 6697 tests and failed on this host's unprovisioned ast-grep hook environment (6659 passed, 37 skipped, 1 warning, 1 error, exit 1), and it emitted the platform obligation `wsl-platform: next checkpoint`, which is undischarged. The exact results are in the validation record filed with the sources. No new full-suite, coverage, live-qualification or activation result is claimed, and no product suite, live run or activation was commissioned.